Tuned Baselines Shrink the Gains: An Honest Evaluation of a Small Learned Congestion Controller in a User-Space Reliable UDP Transport

Harshavardhan Mallela, Rohit Arya, Shloak Sinha
Vellore Institute of Technology, Vellore, India
Code and data: `https://github.com/Capsturrrr/poise`

Abstract

We built Poise, a reliable transport over UDP in C (Selective Repeat, adaptive timers, pluggable congestion control), and asked whether a 5–32–32–5 neural network that picks window multipliers can beat classical control. Against *untuned* AIMD the margin looked large; against *tuned* AIMD (window cap and slow-start threshold raised, same timer floor for every policy) it fell to 15–48% on four of five emulated paths and vanished on a 10%-loss path. We report a methodology (paired seeds, held-out random paths, a virtual-time build of the real sender calibrated against the real clock) and four extra tests motivated by recent literature: paths that change mid-transfer, a BBR-style baseline, fairness with unequal RTTs and against kernel TCP, and interpretability/certification of the network. A simplified BBR beats the agent on one path. On a Linux kernel (tc/netem) the agent is 65–83% faster than tuned AIMD but kernel TCP remains 1.5–1.7× faster. Formal interval-bound certification succeeds only for tiny input perturbations. We also document four measurement mistakes of our own that changed or could have changed conclusions. All results are reproducible with one command.

1 Introduction

Learned congestion control is often reported with large gains over human-designed algorithms, yet recent experimental work finds that the picture is more nuanced [3]. Gains are sensitive to how the baseline is configured, to path dynamics, to fairness, and to whether the controller can be trusted. We study a deliberately small controller in a transport we control end to end, so that every layer can be inspected.

Contributions. (1) An open, tested user-space reliable transport with a library API and a virtual-time mode that runs the *same C sender* against a simulated path. (2) An evaluation recipe in which baselines are tuned before comparison; we show that tuning removes most of the apparent advantage of the learned policy. (3) Tests of dynamic paths, a model-based baseline, fairness (equal and unequal RTT, kernel TCP), robustness to input noise, distillation into a decision tree, and interval-bound certification. (4) Negative results and a log of our own measurement errors.

2 System

Transport. Packets carry a 13-byte header (sequence, ack, flags, CRC-16, length) and up to 512 payload bytes. The default is Selective Repeat: a 256-packet window, ACKs with a 32-byte selective bitmap, per-packet timers, and an adaptive retransmission timeout shared by all policies. A fuzzer fed 10^6 corrupted packets to the parser with none accepted (a first byte-sum checksum let about 2% through, hence CRC-16).

Policies. AIMD (tuned: cap 128, threshold 128), CUBIC [1], a tabular Q-learner, a BBR-style model-based policy [2], and the neural agent. The agent is a 5–32–32–5 ReLU network (1,413 weights) that every clamp(srtt, 100, 500) ms scores five window factors $\{0.5, 0.85, 1, 1.15, 1.5\}$ from RTT growth, loss fraction, window, throughput share and last action. Deployment adds slow start to 64 packets, pacing at 2 cwnd/srtt, and a hand-over to AIMD when interval loss exceeds 10%. It was trained by Deep Q-learning in a Python simulator (cap 32) and generalises to a cap of 128.

Evaluation harness. The sender runs against (i) an in-process emulated path, (ii) a user-space UDP proxy on real sockets, (iii) a virtual-time build (same C code, simulated clock and receiver; matched the real clock in 15 of 15 cells) and (iv) Linux tc/netem on loopback with a token-bucket bottleneck. Comparisons use paired seeds, 95% t intervals and Welch tests; the policy was selected on one seed set and scored on another.

3 Results

Tuned baselines. On five named paths (100 paired seeds, virtual time) the agent needs 15% (5G-like), 48% (2% loss), 42% (satellite-like) and 41% (high bandwidth-delay) less time than tuned AIMD, and is not distinguishable on a 10%-loss path (−8%, n.s.). On 100 random held-out paths its geometric-mean time is 0.53× AIMD's, faster on 89 paths and never more than 20% slower; tuned CUBIC is 0.98×, faster on 51 and >20% slower on 27. An ablation shows that the larger window cap and pacing account for most of the gain; with the agent's original settings it was only about 14% faster than tuned AIMD.

Table 1: Mean completion time (s) of 3,000 packets, 20 paired seeds, virtual time.

Path	AIMD	CUBIC	BBR-style	Neural
5G-like	2.78	7.14	2.26	**2.21**
Lossy 2%	19.79	53.25	**8.18**	9.76
Satellite-like	32.76	26.62	16.78	**11.18**
Fast	7.83	12.45	6.23	**3.72**
Hostile 10%	94.7	208.8	**92.3**	99.4

Table 2: Paths that change mid-transfer: mean time (s) for 4,000 packets, 20 seeds.

Case	AIMD	CUBIC	BBR-style	Neural
Bandwidth drop	13.58	17.74	13.39	**12.72**
Bandwidth rise	9.21	15.10	5.00	**4.32**
Delay jump	33.32	27.54	25.94	**19.80**
Hand-over pattern	12.84	21.40	10.79	**8.18**

Model-based baseline. Table 1 adds a simplified BBR (bandwidth max-filter, min-RTT, pacing-gain cycle, window = 2 BDP, no loss response, ProbeRTT replaced by 10 s min-RTT expiry). It beats tuned AIMD on four paths and is the fastest policy on the 2%-loss path, so the claim "the agent beats the established alternatives" holds on four of five paths, not all. Two defects in our own BBR (a stall at 10% loss after one stale bandwidth sample; 163 s after a delay jump because min-RTT never expired) were found by these tests and fixed. BBRv3 is reported to degrade above ≈2% random loss; our simplified policy does not, so no claim is made about the Linux implementation.

Dynamic paths. A `--dyn` schedule changes rate and delay during a transfer (Table 2). The agent, trained only on constant paths, is fastest in all four cases (6–53% faster than tuned AIMD), at the price of 1–6% retransmissions against 0.5–2% for AIMD. The cases are hand-made, not recorded traces.

Fairness. Two neural flows on a shared real-socket bottleneck split it evenly with equal RTT (Jain 0.97). With the second flow's RTT longer by 150 ms, two neural flows reach Jain 0.84 (short-RTT flow 71%) against 0.64 for two AIMD flows (88%); six pairs per cell, so only the largest gap is clear. Against tuned AIMD the agent takes about 68% of the link. Against a kernel TCP (CUBIC) flow on the same netem bottleneck it takes 24–34% of the link (tuned AIMD 5–17%; three runs per cell), so it does not starve TCP, largely because it is slower than TCP when alone.

Real kernel. Table 3 repeats the comparison on netem with a separate token-bucket bottleneck. The agent is 65–83% faster than tuned AIMD (reordering: not significant, $p = 0.09$, tuned AIMD is bimodal), but *kernel TCP is 1.5–1.7× faster than the agent on every path.* We do not claim to beat TCP.

Interpretability and trust. On about 5,500 visited test states the agent is bang-bang (factor 1.5 in 63% of decisions, 0.5 in 17%); shuffling any one input changes 16–45% of decisions, so no input is redundant. A depth-5 decision tree (32 leaves) matches the network's decision on 79% of unseen states (87% at depth 8), yet reproduces its completion time within the intervals on four of five paths (9% slower on the 5G-like path). Gaussian noise (s.d. up to 0.2) on the measured inputs changes completion time by at most 12%. Interval bound propagation [5] certifies the chosen action for only 43% of states at ±0.001, 8% at ±0.003, 0.2% at ±0.01 and none at ±0.03, while random sampling in the same boxes finds the action stable in 99%, 96%, 83% and 61% of cases: robust in practice, not provable. Training for certification is future work.

Table 3: Linux tc/netem, 500 packets of 512 B, mean time (s), 5 runs (3 on satellite).

Path	TCP	AIMD	Neural	Neural/AIMD
Lossy (2%)	0.88	4.75	1.35	0.29
5G-like	0.28	1.23	0.43	0.35
Satellite	1.91	16.72	2.92	0.17
Reordering	0.94	4.06	1.64	0.40 (n.s.)

4 Threats to Validity and Mistakes

Most numbers are from simulation (virtual time), calibrated against the real clock; the kernel tests are loopback on one laptop with few runs. A single run between two laptops over a relayed internet path (5 repetitions, 2 MB) showed that Poise completes real transfers between machines, but the path drifted and 4 of 20 Poise runs timed out, so it does not rank the policies; kernel TCP was 2 to 6 times faster on the clean repetitions. Tuned baselines are tuned by us; a different tuning could change the outcome. Fairness shares are good to ∼10 points. Our BBR is a simplification. We made four measurement errors, all caught and corrected: (i) netem's queue limit counted packets in flight and overflowed on satellite paths; (ii) the AIMD arm was untuned; (iii) TCP sent 1,024-byte payloads against 512 for Poise; (iv) without a file, the sender emits synthetic 40-byte packets, so Poise carried about 12× fewer bytes than TCP in the first kernel runs (found when a goodput exceeded link capacity). A timer defect that hurt the baseline more than the agent also inflated early margins until fixed. After re-running, the kernel ratios changed little, but we report the errors because they could have decided the result.

5 Related Work

Learned controllers include Remy, PCC-Vivace, Aurora and Orca; BBR [2] and CUBIC [1] are the human-designed references. An in-depth experimental comparison finds learned controllers' advantages less consistent than reported [3]. Fairness generalisation of learned control is studied in Jury [4], and learning with formal certificates in C3 [5]. Our contribution is not a new algorithm but a small, fully reproducible case study that quantifies how much of a learned gain survives tuned baselines, dynamic paths, a model-based competitor,

real-kernel validation and certification.

6 Conclusion

A 1,413-weight network gives a real but smaller-than-advertised gain over tuned AIMD in our transport, is beaten on one path by a BBR-style policy, is level with AIMD at 10% loss, and does not match kernel TCP. It copes with changing paths, is a polite neighbour of kernel TCP, and tolerates input noise, but cannot yet be formally certified. The most reusable result is the methodology: tune the baselines first, test on held-out and dynamic paths, validate on a real kernel, and publish the mistakes.

References

[1] S. Ha, I. Rhee, L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," *ACM SIGOPS OSR* 42(5), 2008.

[2] N. Cardwell et al., "BBR: Congestion-Based Congestion Control," *ACM Queue* 14(5), 2016.

[3] A. Mazilu, B. Giacomoni, I. Parisis, "Learning-Based vs Human-Derived Congestion Control: An In-Depth Experimental Study," arXiv:2510.25105, 2025.

[4] Tian et al., fairness generalizability of learned congestion control (Jury), ACM EuroSys 2025.

[5] Yang et al., "C3: Learning Congestion Controllers with Formal Certificates," arXiv:2412.10915, 2024.